Ore-acle: A Multimodal RAG System for Complex Game Wikis with Verifiable Citations

James Michael Gampper
Faculty of Arts
Chulalongkorn University
jgampperjr@gmail.com

Abstract

We present **Ore-acle**, an offline multimodal retrieval-augmented generation (RAG) system for complex game wikis, using Minecraft as a case study. In this domain, relevant knowledge is spread across hierarchical sections, infoboxes, crafting grids, and image-heavy pages, so retrieval quality depends not only on the retriever itself but also on how the corpus is processed. We examine three custom interventions in this setting: wiki-specific section-aware chunking, tuned SQLite FTS5 keyword retrieval, and domain-specific extraction of Minecraft UI recipe grids. The system processes 12,487 HTML wiki pages into 94,404 section-aware chunks and 61,248 local images, and is evaluated on 333 LLM-generated but human-curated question-answer pairs. Across ablations over search mode, RRF weighting, embedding model, chunking strategy, and generator choice, pure semantic retrieval achieves the highest MRR, while a hybrid configuration with tuned sparse search remains the best operational tradeoff for multimodal grounding. A cross-encoder reranker ablation over five configurations finds that certain rerankers can improve retrieval quality, while weaker rerankers may decrease retrieval quality. These results suggest that effective RAG for technical wikis depends as much on preprocessing and post-retrieval reranking as on model choice.

1 Introduction

1.1 Motivation

Complex sandbox games like Minecraft (**?**) involve thousands of interlocking mechanics, crafting recipes, entity behaviors, and version-specific rule changes. Players routinely rely on wikis to answer questions that require exact values, cross-edition distinctions, or visual-spatial knowledge such as crafting layouts. Large Language Models (LLMs) offer a more natural interface to this information, but in this domain they frequently hallucinate specifics, do not provide sources, and collapse visual structure into unreliable paraphrase.

1.2 Problem Statement

Building a high-quality wiki RAG system for this domain is difficult for three immediate system reasons:

- **Verifiability**: answers must be traceable to exact source spans, not just whole pages.
- **Structured content**: crucial facts are stored in infoboxes, tables, and Minecraft UI recipe grids rather than plain text.
- **Retrieval mismatch**: naive chunking and default keyword-search semantics perform poorly on natural-language technical queries.

1.3 Our Solution: Ore-acle

We develop **Ore-acle Offline**, a local-first RAG system for Minecraft Wiki question answering. Ore-acle combines ChromaDB dense retrieval with a tuned SQLite FTS5 keyword index, grounds responses with chunk-level inline citations, and preserves visual context through local image retrieval and structured extraction of crafting grids. It is both a working assistant for end users and a testbed for studying which custom engineering decisions materially improve retrieval quality in a multimodal technical corpus.

1.4 Contributions

1. **Wiki-specific retrieval design**: a section-aware chunker that filters navigation sections, preserves structured blocks, and merges orphan chunks using an absorb-backward heuristic.

2. **Tuned sparse retrieval for technical queries**: a custom SQLite FTS5 configuration with OR semantics, stopword filtering, and title-weighted BM25 that substantially improves the keyword retrieval baseline.

3. **Domain-specific structured extraction**: an MCUI parser that converts CSS-based crafting and recipe layouts into text-searchable spatial representations.

4. **Benchmark and ablation suite**: a 333-question benchmark, plus ablations over retrieval mode, fusion weight, embedding model, chunking strategy, reranker, and generator choice.

2 Related Work

2.1 Retrieval-Augmented Generation

? survey dense, sparse, and hybrid RAG architectures, while **?** highlight the difficulty of measuring grounding and citation faithfulness. **?** distill retrieval best practices on general-domain corpora. Our setting differs in that the main bottlenecks are not only retriever choice, but also domain-specific preprocessing and evaluation design.

2.2 Domain-Specific RAG

RAG has proven valuable in domains where factual precision is mandatory, including medicine (**??**) and law (**??**). These settings share several properties with game wikis: rapidly changing technical details, structured source material, and costly hallucinations. To our knowledge, however, prior work has not treated gaming wikis as a distinct multimodal retrieval domain with its own preprocessing and evaluation requirements.

2.3 Minecraft as AI Testbed

Minecraft is already a useful AI benchmark for open-ended interaction and embodied learning (**???**). That work focuses on action grounding, whereas Ore-acle targets *information* grounding: retrieving and citing exact technical knowledge from a large wiki rather than learning behavior through exploration.

2.4 Citation in Language Models

Google's NotebookLM (**?**) introduced verifiable inline citations for research documents, establishing UI patterns for source attribution. We adapt this paradigm to technical game documentation.

2.5 Hybrid Search Architectures

Reciprocal Rank Fusion (RRF) (**?**) remains the standard for combining dense and sparse retrieval signals. However, its application to highly structured technical domains remains underexplored. **?** demonstrates lightweight RAG for STEM questions but does not address multimodal requirements.

2.6 Chunking Strategies

? evaluates text splitting techniques for RAG, finding that semantic coherence in chunks significantly impacts retrieval quality. Wiki-specific challenges, particularly preserving infobox tables and hierarchical section structures, require domain-adapted approaches.

3 The Minecraft Wiki Domain

3.1 Domain Characteristics

The Minecraft Wiki documents a procedurally generated world governed by thousands of interlocking systems, including biome generation rules, crafting recipes, mob behavior, redstone logic, and version-specific patch behaviors. Several properties make it an unusually demanding RAG testbed:

Figure 1: A player-built structure from the 10th Anniversary Celebration map, illustrating the spatial and visual complexity characteristic of Minecraft environments. Answering questions about structures, biomes, or mechanics often requires images alongside text.

Figure 2: A crafting recipe grid (e.g., Daylight Detector), showing the precise ingredient requirements needed to construct items. A 1-cell shift or incorrect material variant produces an entirely different item or fails silently; this scale of structured visual-spatial knowledge is ill-suited to parametric LLM recall.

- **Visual-spatial reasoning**: Many answers are incomprehensible without image context (e.g., redstone circuit diagrams, biome terrain features, mob spawn conditions).
- **Recipe breadth**: Hundreds of crafting recipes with exact positional constraints; a 1-cell shift produces an entirely different item or fails silently.
- **Version fragmentation**: Mechanics, item IDs, and behaviors differ substantially between Java Edition, Bedrock Edition, and legacy console releases. A query without version context is genuinely ambiguous.
- **Encyclopedic depth**: The wiki spans 12,487 articles covering biomes, entities, blocks, items, structures, mechanics, commands, and changelogs, far exceeding the context window of any single LLM call.

3.2 Query Complexity and Scale

The 333-pair evaluation set spans three difficulty tiers. Easy questions target single-fact lookups; medium questions require synthesising information across infoboxes or related sections; hard questions demand cross-mechanic or cross-edition reasoning. This last category is where purely parametric LLMs fail most often, because the answer typically depends on exact numeric or version-conditional details scattered across multiple wiki sections.

3.3 Failure Modes of Existing Solutions

General-purpose LLMs fail in this domain by confabulating specifics, flattening visual structure, and missing post-pretraining game changes. Ore-acle addresses these failure modes by grounding generation in retrieved chunks, preserving structured wiki content during preprocessing, and attaching precise inline citations and images to each response.

4 System Architecture

4.1 Offline Pipeline

Ore-acle ingests 12,487 HTML pages from the Minecraft Wiki, producing 94,404 section-aware chunks, 78,172 LangChain comparison chunks, and 61,248 locally stored images. Figure **??** summarizes the end-to-end offline pipeline. The key design choice is that preprocessing is not treated as a generic cleaning stage: it is where much of the downstream retrieval quality is won or lost.

Two pipeline decisions are especially important. First, the text cleaner preserves infoboxes and extracts Minecraft UI recipe widgets directly from CSS grids into text-searchable representations such as `[Crafting Recipe: [., ., .] [Diamond, Book, Diamond] ...]`. This allows both dense and sparse retrievers to index crafting layouts that would otherwise remain buried in presentational HTML. Second, the chunker is explicitly wiki-aware: it preserves list blocks, splits text at sentence boundaries, removes pure navigation sections, and merges sub-threshold orphan chunks backward into the previous content chunk when this stays within budget. These heuristics reduce ranking noise and keep technical details attached to the local context that makes them interpretable.

For all main experiments, we use `nomic-ai/nomic-embed-text-v1.5` (**?**) as the default dense encoder because it wins the embedding ablation on both text retrieval and image recall. BAAI/bge-m3 (**?**) and multilingual-E5-Large (**?**) are evaluated separately in Section **??**.

4.2 Retrieval Architecture

Storage Strategy: ChromaDB stores dense embeddings for the section-aware and LangChain chunk collections; SQLite FTS5 stores the full keyword index over section-aware chunks; and the local filesystem stores WebP images referenced by hash-based filenames.

Runtime Query Flow:

The dense retriever alone is strong, but Ore-acle's main systems contribution is making sparse retrieval competitive enough to help rather than harm fusion. Out-of-the-box FTS5 fails on natural-language

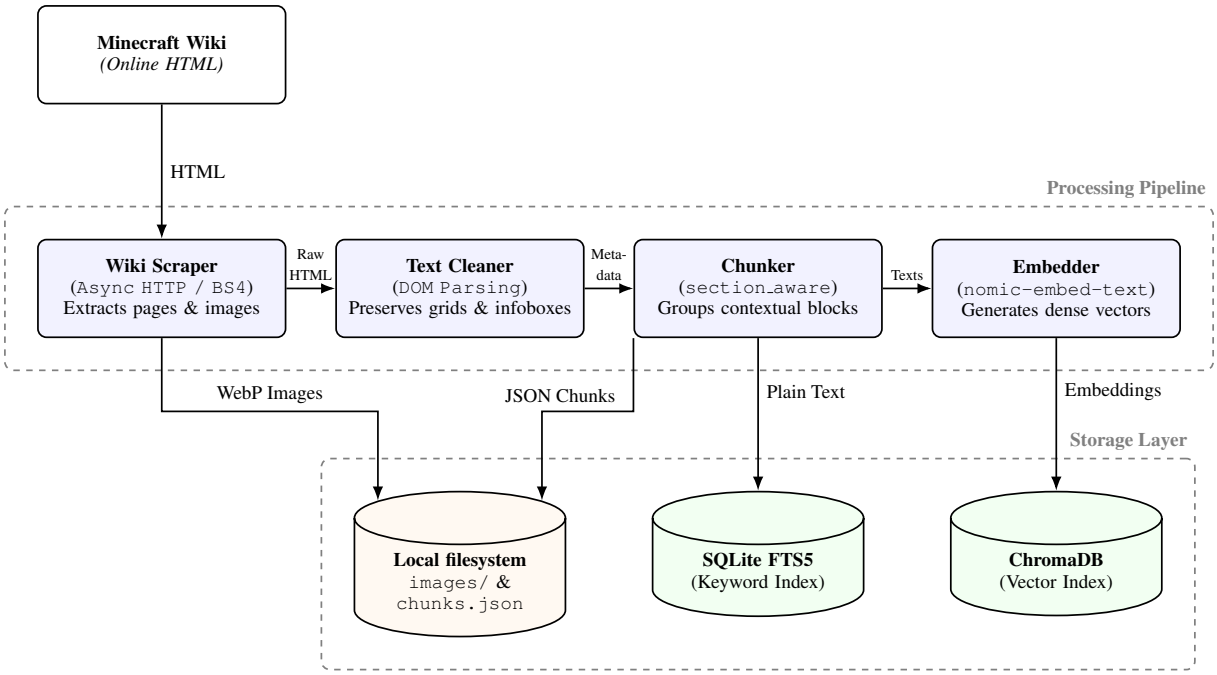

Figure 3: Ore-acle Offline Data Pipeline: From live Minecraft Wiki to hybrid local storage.

technical questions because player-phrased queries rarely match wiki text verbatim: the query "how do I breed pigs" would require a wiki sentence containing all four words, which does not exist. We therefore rewrite queries into OR-connected term sets, remove hardcoded stopwords, and apply title-weighted BM25 scoring so exact article-title matches receive much higher priority than body-only matches. This tuned keyword channel raises the sparse baseline from MRR 0.425 to 0.515 and is what makes weighted RRF useful rather than cosmetic.

Ore-acle uses chunk-level provenance rather than page-level citations. Each retrieved chunk carries page title, section heading, page URL, and the exact text span shown to the model. The final answer cites these chunks inline with `[n]` markers, allowing a reader to inspect a specific supporting section instead of an entire wiki page. This design choice is central to the high citation-faithfulness scores reported later, and it makes factual inspection practical for technical wiki questions.

4.3 Generation with Citations

Responses use NotebookLM-style inline citations with verbatim grounding, explicit page titles, and original URLs. When relevant images are available, the system injects them alongside the answer so that recipe layouts, structures, and other visual mechanics are not reduced to plaintext alone.

4.4 UI Walkthrough & Multimodal Capabilities

The frontend is a lightweight chat interface whose main research role is to demonstrate grounded output rather than novel interaction design. Users may provide screenshots as additional multimodal context, which are appended to the prompt alongside retrieved wiki chunks. Figure **??** illustrates the resulting behavior: the system answers a gameplay question with inline citations and an explicitly rendered crafting layout instead of forcing the model to paraphrase a recipe from memory.

5 Experimental Setup

5.1 Dataset

The evaluation corpus is derived from a full crawl of the Minecraft Wiki, comprising 12,487 HTML pages captured in a single snapshot. Pages were cleaned, section-split, and chunked into 94,404 segments of up to 512 tokens with a 50-token overlap, using a recursive section-aware splitter that preserves infobox

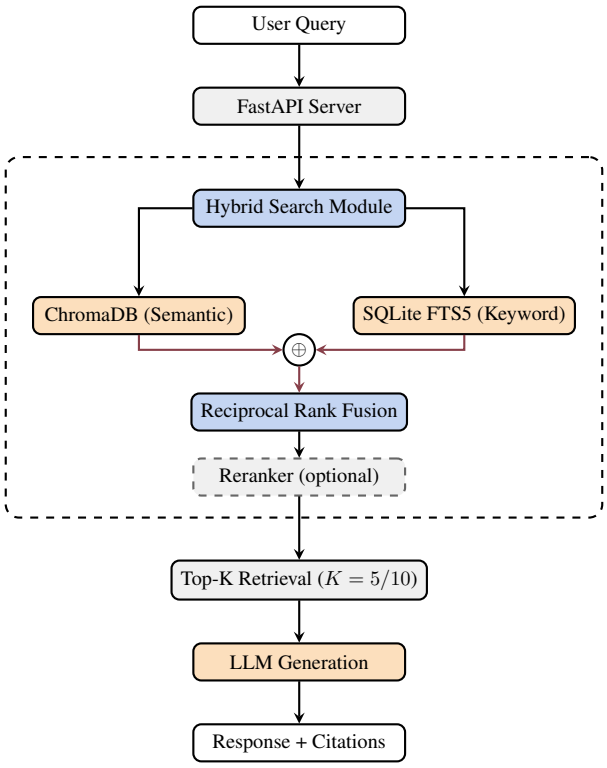

Figure 4: Runtime hybrid search pipeline with optional cross-encoder reranker applied post-RRF fusion.

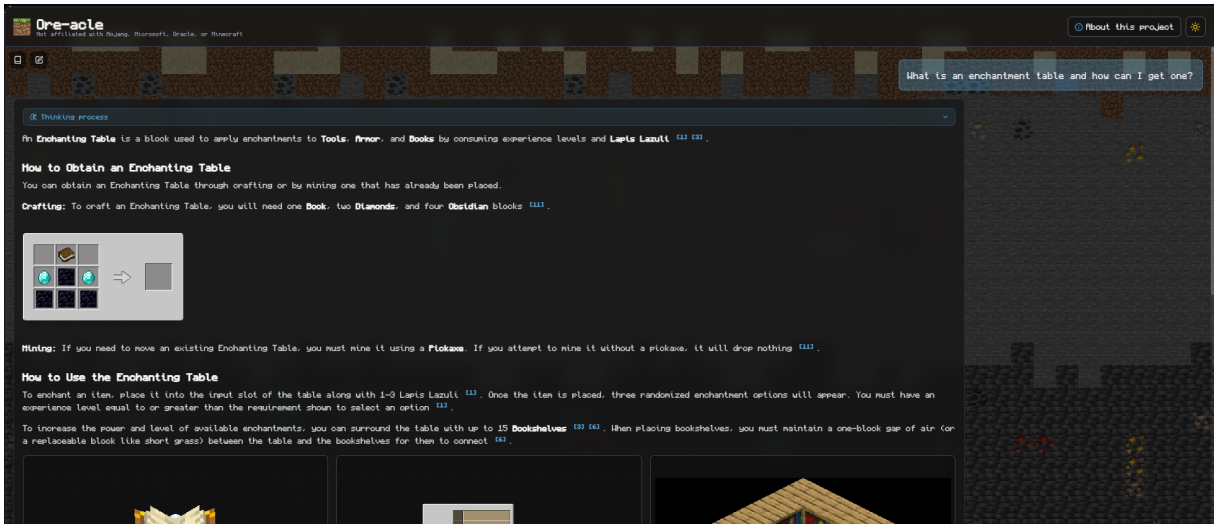

Figure 5: Ore-acle UI response to the query "What is an enchantment table and how can I get one?" The system retrieves the relevant wiki chunks and renders the crafting recipe grid (Book, two Diamonds, four Obsidian) inline within the response, alongside inline citations. This demonstrates the system's multimodal grounding: structured recipe data extracted from wiki CSS grids is presented visually rather than as raw text.

table boundaries. Concurrently, 61,248 images were downloaded, decoded to WebP, and stored under content-addressed filenames derived from their source URLs.

The golden test set consists of 333 question-answer pairs generated via Gemini Flash Lite (**?**). One hundred twenty source pages were selected using a PageRank-plus-content-density heuristic so that the benchmark emphasizes high-value, well-linked wiki pages rather than uniformly random articles. For each page, three questions were generated at easy, medium, and hard difficulty, then manually reviewed. Additional adversarial prompts were seeded to probe ambiguous or narrative questions outside straightforward mechanics lookups.

5.2 Evaluation Framework

[1]

Golden Test Set Generation:

- 333 question-answer pairs spanning explicit game mechanics, lore, and structural challenges.
- Generated via Gemini Flash Lite (OpenRouter) with a two-pass image verification protocol.
- Inclusion of manually seeded adversarial/narrative edge cases (e.g., "Who is Steve?", "Best gearsets") to secure generation robustness against trivia blindspots.
- Streaming JSON processing via `ijson` applied for localized memory parsing efficiency.

Two-Phase Ablation Protocol:

Table 1: Ablation study design

Phase	Axis	Configurations
1: Retriever	Search Mode	Semantic / Keyword / Hybrid
	Embedding	BAAI/bge-m3 / Nomic / E5-Large
	Chunking	Section-aware / LangChain
	Reranker	No reranker / BGE v2-m3 / Qwen3-0.6B / Qwen3-4B / zerank-2
2: Generator	LLM	Gemma 4 e2B / e4B / 31B / Gemini Flash Lite

5.3 Metrics

Table 2: Evaluation metrics

Metric	Description
Recall@5/10	Proportion of relevant chunks in top-K
Precision@10	Accuracy of top-10 retrieved chunks
MRR	Mean Reciprocal Rank of first relevant result
Image Recall	Proportion of questions with correct image retrieval
Token F1	Lexical overlap between generated and gold answer
ROUGE-L	Longest common subsequence for answer quality

5.4 Baselines

All retrieval configurations are evaluated relative to a semantic-only dense retriever using `nomic-embed-text-v1.5`. Keyword-only search provides a sparse lower bound, and the hybrid RRF configuration sweeps $\alpha \in \{0.0, 0.2, \ldots, 1.0\}$ to quantify how much sparse evidence should influence the final ranking.

[1] Evaluations tracking latency were conducted on consumer hardware (Intel Core i5-12450H, 16 GB DDR4, NVIDIA RTX 4060 8 GB VRAM) entirely locally to validate baseline accessibility.

6 Results and Ablation Studies

6.1 Search Mode Ablation

Configuration: 333-pair questionset, nomic-embed-text-v1.5 embeddings, section-aware chunking

Table 3: Search mode comparison. Best scores in bold.

Mode	MRR	R@5	R@10	Img Rcl
Semantic	**0.625**	**0.458**	**0.514**	**0.290**
Keyword (OOTB, length-filter)	0.425	0.358	0.465	0.102
Keyword (Custom BM25)	0.515	0.407	0.490	0.131
Hybrid (RRF, α=0.80)	0.612	0.449	0.516	0.265

Key Finding: The main result of Table **??** is not simply that semantic retrieval is strong; it is that sparse retrieval only becomes competitive after domain-specific tuning. Replacing default FTS5 behavior with OR semantics, stopword filtering, stemming, and heavy title weighting raises keyword MRR from 0.425 to 0.515, a 21% relative gain. Pure semantic retrieval still achieves the best MRR (0.625), but hybrid search remains the operational choice because it preserves near-peak text quality while maintaining stronger multimodal grounding than either sparse baseline.

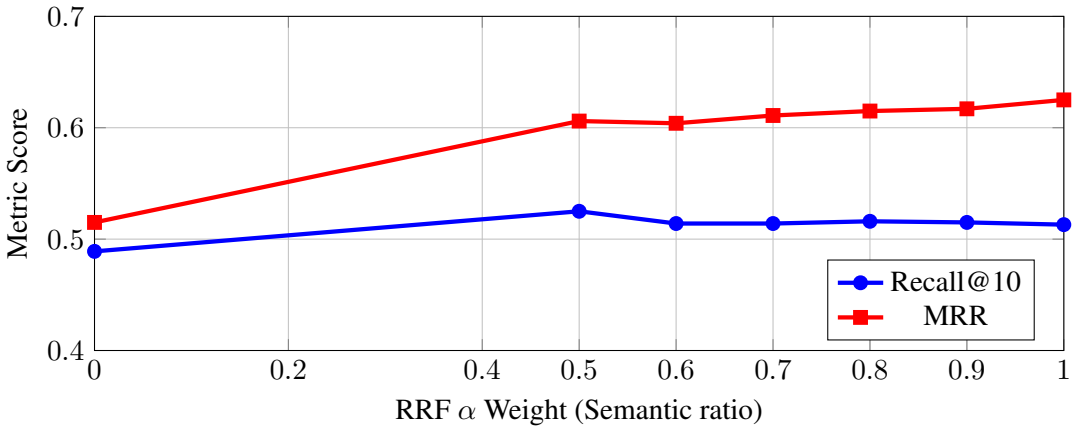

Figure 6: Effect of RRF α parameter on Retrieval Quality (333-question set). $\alpha = 0$ represents pure keyword, while $\alpha = 1$ is pure semantic search. $\alpha = 0.80$ achieves the best Recall@10 among hybrid configurations while maintaining near-peak MRR.

The α sweep shows that the dense retriever should dominate fusion, but not entirely. Moving from $\alpha = 0$ to $\alpha = 0.80$ yields a steep gain in MRR, after which returns diminish; the remaining keyword contribution is still useful for article-title matches and exact lexical cues that help multimodal retrieval. This explains why Ore-acle locks α=0.80 (80% semantic weight, 20% keyword) rather than defaulting to pure semantic search in deployment.

6.2 Embedding Model Ablation

Configuration: 333-pair questionset, section-aware chunking, hybrid search (α=0.80, K=20)

Table 4: Embedding model comparison. Best scores in bold.

Model	Dim	MRR	R@5	R@10	Img Rcl
`nomic-embed-text-v1.5`	768	**0.612**	**0.455**	**0.519**	**0.265**
`BAAI/bge-m3`	1024	0.605	0.454	0.512	0.225
`multilingual-e5-large`	1024	0.508	0.371	0.432	0.141

Key Findings: `nomic-embed-text-v1.5` wins on every reported retrieval metric, so it remains the default model for all subsequent ablations. `bge-m3` is close on text metrics but weaker on image recall; `multilingual-e5-large` trails substantially, suggesting that its multilingual objective is not the right fit for this English-only technical corpus.

6.3 Chunking Strategy Ablation

Preserving semantic boundaries is especially important in wiki corpora, where a short subsection often only makes sense in the context of the section directly above it. Our section-aware chunker therefore does more than respect headings: it preserves list blocks as atomic units, performs sentence-boundary overlap for prose, skips pure navigation sections that pollute ranking, and merges sub-threshold chunks back into the previous content chunk when possible. This absorb-backward rule is designed for common wiki artifacts such as short trailing subsections, "See also" stubs, and fragmented mechanic notes. We compare this strategy against LangChain `RecursiveCharacterTextSplitter` under the same embedding and hybrid-retrieval configuration.

Table 5: Chunking strategy comparison (nomic-ai/nomic-embed-text-v1.5, hybrid α=0.80). Best scores in bold.

Strategy	MRR	R@5	R@10	Img Rcl
Section-Aware	**0.636**	**0.488**	**0.572**	**0.279**
LangChain†	0.614	0.462	0.517	0.203

Key Finding: The custom section-aware chunking strategy effectively retains wiki article structure, generating semantically coherent chunks that achieve comparable text retrieval to LangChain while providing superior image grounding. Section-aware chunking is the definitive choice for wiki RAG. We note that this comparison is holistic; isolating the contribution of individual heuristics (navigation filtering, absorb-backward merging, list preservation) remains future work.

6.4 Reranker Ablation

Configuration: 333-pair questionset, nomic-embed-text-v1.5, section-aware chunking, hybrid search (α=0.80). All rerankers are applied post-RRF over a widened candidate pool of 30 chunks, re-ranking to the final top-10. Latency is reported per query on consumer hardware (RTX 4060 8 GB VRAM, Intel Core i5-12450H).

Table 6: Reranker comparison (hybrid, nomic-embed-text-v1.5, section-aware, 333 questions). PassR@10 = passage-level span recall at 10. Best scores in bold.

Reranker	MRR	R@5	R@10	Img Rcl	PassR@10	Latency
No reranker	0.612	0.452	0.518	0.265	0.508	0.2 s
BGE Reranker v2-m3	0.604	0.456	0.520	0.230	0.535	3.7 s
Qwen3-Reranker-0.6B	0.604	0.448	0.526	**0.317**	0.542	2.0 s
Qwen3-Reranker-4B	**0.636**	0.472	0.543	**0.317**	**0.568**	33.0 s
zerank-2	**0.636**	**0.483**	**0.571**	0.203	0.547	13.3 s

Key Findings: Reranking improves retrieval quality materially only when the reranker is sufficiently capable. BGE Reranker v2-m3 *decreases* MRR from 0.612 to 0.604 and image recall from 0.265 to 0.230, suggesting its scoring function does not align well with Minecraft-specific technical query semantics. Qwen3-Reranker-0.6B recovers recall (R@10=0.526) at only 2.0 s latency and uniquely achieves the highest image recall (0.317), making it an attractive lightweight option when multimodal grounding is the priority. Qwen3-Reranker-4B and zerank-2 tie on MRR (0.636), but zerank-2 substantially outperforms on Recall@10 (0.571 vs. 0.543) at less than half the latency (13.3 s vs. 33.0 s). zerank-2 is therefore the recommended post-retrieval component for deployments where text retrieval quality and interactive latency both matter.

6.5 Generator Evaluation

Configuration: 333-pair questionset, nomic-embed-text-v1.5 embeddings, section-aware chunking, hybrid search (α=0.80). All models were prompted with identical system instructions requiring inline `[n]` citations. We evaluate four models: Gemma 4 e2B, Gemma 4 e4B, and Gemma 4 31B (**?**), alongside Gemini Flash Lite (**?**). We purposefully select this spread to evaluate whether highly-grounded retrieved context allows ultra-small, edge-deployable models (e2B/e4B parameters) to punch above their weight against state-of-the-art frontier models.

Table 7: Generator evaluation (333-question set). CitF = Citation Faithfulness. Latency excluded: Ollama (local) and OpenRouter (API) are not comparable on this axis. Best scores in bold.

Model	Backend	Avg F1	ROUGE-L	BERTScore	CitF
Gemma 4 e2B	Ollama (local)	0.244	0.163	0.711	0.634
Gemma 4 e4B	Ollama (local)	0.108	0.074	0.310	0.295
Gemma 4 31B	OpenRouter (API)	**0.290**	**0.174**	0.844	0.973
Gemini Flash Lite	OpenRouter (API)	0.288	0.168	**0.844**	**0.983**

Key Finding: Once retrieval is well tuned, generator choice matters less than expected for factual grounding. The strongest cloud models tie on BERTScore and achieve near-perfect citation faithfulness, while Gemma 4 e2B remains surprisingly viable given its size. Gemma 4 e4B's low average (F1=0.108) reflects a high rate of empty responses rather than uniformly poor quality; when it does answer, its responses are factually grounded (F1 0.32–0.60). This likely stems from finetuning brittleness with the strict inline-citation format. Overall, retrieval and provenance design, rather than sheer generator scale, are the primary determinants of answer reliability in this setting.

6.6 Qualitative Examples

We keep two representative examples in the main paper: one factual lookup that benefits from structured grounding, and one hard cross-edition query that tests whether the system can preserve subtle mechanic distinctions.

Easy: Factual Drop Query

Query: *"What does a Blaze drop?"*

Ore-acle retrieves nine chunks from the *Blaze* and *Blaze Powder* wiki pages and produces a structured response with inline citations and a programmatically generated Looting table:

```
When killed by a player or a tamed wolf, a Blaze drops:

- Blaze Rod [1][4]: drop rate increases with Looting.
  | Looting Level | Expected Drops |
  | None          | 0.50           |
  | Looting I     | 0.75           |
  | Looting II    | 1.00           |
  | Looting III   | 1.25           |

- Experience Orbs: 10 XP per kill [1].

Blaze Rods are a primary ingredient for Blaze Powder,
required to fuel Brewing Stands and craft Eyes of Ender [6].
The "Into Fire" advancement is associated with their
acquisition [7][9].
```

The model correctly produces version-specific historical trivia (Legacy Console glowstone dust drops) attributed to the correct source chunk, demonstrating that fine-grained infobox knowledge is retrieved and cited rather than hallucinated.

Hard: Cross-Edition Mechanic Reasoning

Query: *"How does the Bad Omen effect change raid scaling?"*

This question requires reconciling Java and Bedrock Edition semantics, a class of query where parametric LLMs frequently confabulate. Ore-acle retrieves ten chunks spanning the *Bad Omen*, *Ominous Bottle*, *Raid*, and *Illager* pages, and explicitly partitions its answer by edition:

```
Java Edition:
- Higher Bad Omen levels increase raider enchantment
  probability [1][2][4][9].
- Bad Omen II or higher triggers an additional raid wave [1][4].
- Hero of the Village effect level matches the triggering
  Bad Omen level [1][10].

Bedrock Edition:
- Bad Omen level has no effect on raid scaling [1].

Note: Bad Omen affects Trial Spawners (converting them to
Ominous Trial Spawners), but the level does not change trial
difficulty [1][4].
```

The Java/Bedrock asymmetry (higher Bad Omen is meaningless on Bedrock) is precisely the kind of technical distinction that general LLMs often conflate. Ore-acle correctly surfaces the `Bad Omen/Before 1.21` chunk to capture the historical context of the mechanic's redesign in Java Edition 1.21.

7 Discussion

7.1 Keyword Parameter Tuning within Technical Domains

The SQLite FTS5 index uses OR-connected query semantics rather than exact-match, a configuration choice with significant recall impact. Under naive exact-match semantics, multi-word natural language queries return near-zero results because the wiki corpus does not contain verbatim question phrasing. Switching to OR semantics, while filtering single-character tokens as stopwords, raised keyword Recall@10 from effectively 0 to 0.490 (Table **??**), enabling meaningful comparison with the dense retriever. This highlights that keyword retrieval in structured technical corpora requires query-side adaptation: the precision/recall tradeoff is not symmetric with general-domain BM25 assumptions.

7.2 Design Implications for Wiki RAG

Winning Configuration: Hybrid search (α=0.80, nomic-embed-text-v1.5, section-aware chunking) with zerank-2 reranker. This configuration achieves MRR=0.636, R@10=0.571, and Image Recall=0.203

Four design lessons emerge from the ablations. First, domain-specific preprocessing matters: navigation filtering, chunk merging, and recipe-grid extraction materially improve what the retriever can index. Second, sparse retrieval should not be treated as an untuned baseline; in structured technical corpora, query semantics and field weighting determine whether keyword search is useful at all. Third, reranker selection is non-trivial: a standard cross-encoder (BGE Reranker v2-m3) *hurts* MRR in this domain, while only sufficiently capable models (Qwen3-4B, zerank-2) deliver meaningful gains over the unranked baseline. Fourth, image recall and text retrieval trade off across configurations—the reranker that maximises Recall@10 (zerank-2) is not the one that maximises image recall (Qwen3-Reranker-0.6B) so the operational choice depends on the application's modality requirements.

References

G. V. Cormack, C. L. A. Clarke, and S. Buettcher. Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. *SIGIR '09*, pages 758–759, 2009.

Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997*, 2023.

Google. NotebookLM: AI Research Tool with verifiable inline citations, 2023. https://notebooklm.google

A. Kshirsagar. Enhancing RAG Performance Through Chunking and Text Splitting Techniques. *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, 10:151–158, 2024.

R. Kumar. Wikipedia-Savvy-RAG: A Lightweight Retrieval-Augmented Generation System for STEM Question. In *Proc. Int. Conf. Recent Advancements in Artificial Intelligence*, 2025.

S. Li, L. Stenzel, C. Eickhoff, and S. A. Bahrainian. Enhancing Retrieval-Augmented Generation: A Study of Best Practices. *arXiv:2501.07391*, 2025.

H. Yu et al. Evaluation of Retrieval-Augmented Generation: A Survey. *arXiv:2405.07437*, 2024.

J. Chen et al. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv:2402.03216*, 2024.

Google. Gemini: A Family of Highly Capable Multimodal Models. *arXiv:2312.11805*, 2023.

Google. Gemma: Open Models Based on Gemini Research and Technology. *arXiv:2403.08295*, 2024.

Z. Nussbaum et al. Nomic Embed: Training a Reproducible Long Context Text Embedder. *arXiv:2402.01613*, 2024.

L. Wang et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv:2212.03533*, 2022.

D. Han and M. Han. Unsloth: 2–5× faster, 80% less memory LLM finetuning. https://github.com/unslothai/unsloth, 2024.

S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. Lawrence Zitnick, and D. Parikh. VQA: Visual Question Answering. In *ICCV*, pages 2425–2433, 2015.

A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. In *ICML*, volume 139, pages 8748–8763, 2021.

W. Chen et al. MuRAG: Multimodal Retrieval-Augmented Generator for Open Question Answering over Images and Text. In *EMNLP*, pages 5558–5570, 2022.

Y. Chang et al. WebQA: Multihop and Multimodal QA. In *CVPR*, pages 16474–16483, 2022.

K. Singhal et al. Large Language Models Encode Clinical Knowledge. *Nature*, 620:172–180, 2023.

A. Nentidis, G. Katsimpras, E. Vandorou, A. Krithara, L. GaldigaLopez, M. Krallinger, and G. Paliouras. BioASQ at CLEF2023: The eleventh edition of the large-scale biomedical semantic indexing and question answering challenge. In *ECIR*, 2023.

N. Guha et al. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. In *NeurIPS*, 2023.

S. Wu et al. BloombergGPT: A Large Language Model for Finance. *arXiv:2303.17564*, 2023.

L. Fan et al. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. In *NeurIPS*, 2022.

G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv:2305.16291*, 2023.

S. Cai et al. GROOT: Learning to Follow Instructions by Watching Gameplay Videos. *arXiv:2310.08235*, 2023.

Mojang Studios. Minecraft [Video game]. Microsoft Studios, 2011. https://www.minecraft.net

A Full Hyperparameter Tables

Table **??** lists all hyperparameters fixed across the retriever ablation experiments. Table **??** lists the generation parameters held constant during the generator evaluation.

Table 8: Retriever hyperparameters. All values held constant except where explicitly varied per ablation axis.

Parameter	**Value**	**Notes**
Semantic Retrieval (ChromaDB)		
Embedding model	nomic-embed-text-v1.5	Local execution
Candidates per query (K_s)	20	
Keyword Retrieval (SQLite FTS5)		
Query semantics	OR	Phrased as natural-language questions
Stopword filter	tokens ≤ 1 char	
Candidates per query (K_k)	20	
Hybrid Fusion (RRF)		
RRF constant k	20	
Semantic weight α	0.80	Locked after sweep
Final top-K returned	10	
Chunking (Section-Aware)		
Max chunk size	2000 characters	$\approx$ 512 tokens
Overlap	200 characters	$\approx$ 50 tokens
Infobox handling	Preserved as-is	
Image metadata binding	URL-hash join	